



UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT



Data Structure and Algorithm

Laboratory Activity No. 13

Tree Algorithm

Submitted by:
Gabijan, Rhovic M.

Instructor:
Engr. Maria Rizette H. Sayo

November 3, 2025

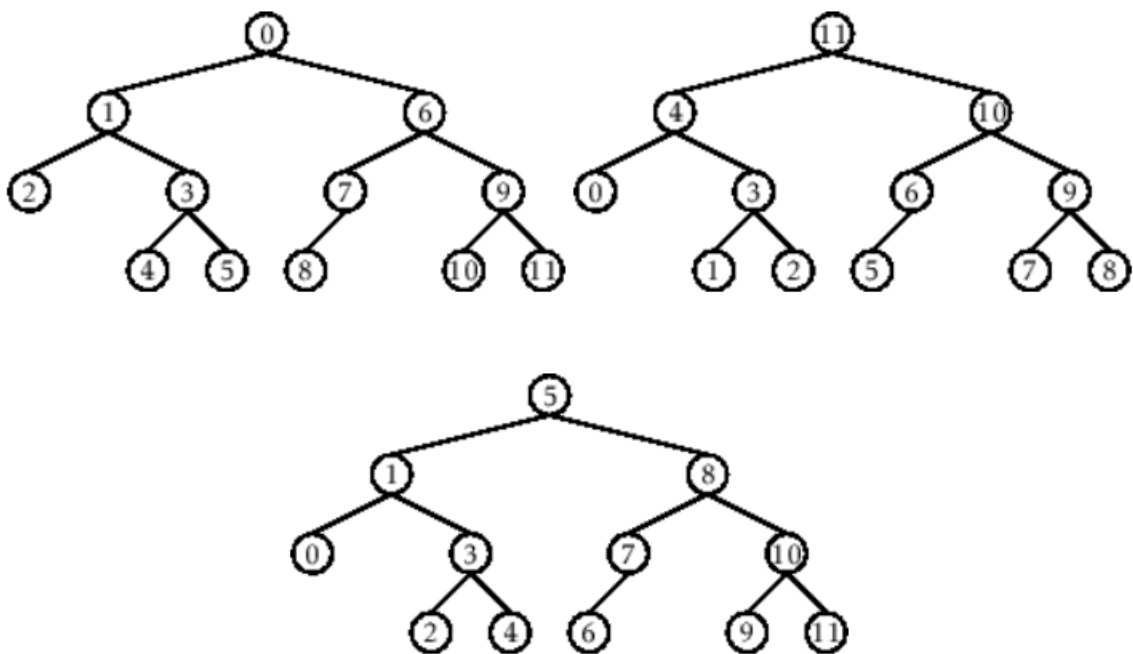
I. Objectives

Introduction

An abstract non-linear data type with a hierarchy-based structure is a tree. It is made up of links connecting nodes (where the data is kept). The root node of a tree data structure is where all other nodes and subtrees are connected to the root.

This laboratory activity aims to implement the principles and techniques in:

- To introduce Tree as Non-linear data structure
- To implement pre-order, in-order, and post-order of a binary tree



- Figure 1. Pre-order, In-order, and Post-order numberings of a binary tree

II. Methods

- Copy and run the Python source codes.
- If there is an algorithm error/s, debug the source codes.
- Save these source codes to your GitHub.
- Show the output

1. Tree Implementation

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []

    def add_child(self, child_node):
        self.children.append(child_node)

    def remove_child(self, child_node):
        self.children = [child for child in self.children if child != child_node]
```

```

def traverse(self):
    nodes = [self]
    while nodes:
        current_node = nodes.pop()
        print(current_node.value)
        nodes.extend(current_node.children)

def __str__(self, level=0):
    ret = " " * level + str(self.value) + "\n"
    for child in self.children:
        ret += child.__str__(level + 1)
    return ret

# Create a tree
root = TreeNode("Root")
child1 = TreeNode("Child 1")
child2 = TreeNode("Child 2")
grandchild1 = TreeNode("Grandchild 1")
grandchild2 = TreeNode("Grandchild 2")

root.add_child(child1)
root.add_child(child2)
child1.add_child(grandchild1)
child2.add_child(grandchild2)

print("Tree structure:")
print(root)

print("\nTraversal:")
root.traverse()

```

Questions:

- 1 When would you prefer DFS over BFS and vice versa?
- 2 What is the space complexity difference between DFS and BFS?
- 3 How does the traversal order differ between DFS and BFS?
- 4 When does DFS recursive fail compared to DFS iterative?

III. Results

1. Depth-First Search (DFS) is preferred when you want to explore deep path first since it reaches deep nodes quickly. It is also useful when the memory is limited because it requires less space. In difference with that, Bread-First Search (BFS) is preferred when finding the shortest path, It is ideal for problem with minimum steps, shortest route, or layered structure.
2. DFS typically has lower space complexity because it only needs to store nodes along the current path and visited nodes. BFS require significantly more memory because it must store all nodes in the current level before moving to the next.
3. DFS explores a graph by moving as deep as possible along one path before backtracking, prioritizing depth over breadth. This results in a traversal

that follows long chains of connections before returning to explore alternative routes. Meanwhile, BFS processes nodes level by level, visiting all immediate neighbors of a starting node before moving outward. This creates a traversal pattern that spreads evenly across each layer of the graph, ensuring that the closest nodes are visited first.

4. DFS recursive can fail when the graph is extremely deep, causing a stack overflow due to exceeding the maximum recursion depth. This is common in graphs with long linear chains or when exploring large datasets. The iterative version, however, avoids this issue by using an explicit stack in memory rather than relying on the system's call stack, making it safer for deep or complex graphs. As a result, iterative DFS is more robust for large-scale or unpredictable graph structures.

IV. Conclusion

Understanding DFS and BFS provides valuable insight into how different graph traversal strategies work and when each one is most effective. DFS allows deep exploration with lower memory usage, making it suitable for large, sparse, or depth-focused problems, while BFS excels in finding shortest paths and exploring graphs level by level. Their differences in traversal order, memory demands, and behavior under recursion highlight the importance of choosing the right approach based on the structure and needs of the problem. Overall, mastering both algorithms equips learners with essential tools for efficient graph processing and problem-solving in computer science.

References

- [1] Co Arthur O.. “University of Caloocan City Computer Engineering Department Honor Code,” UCC-CpE Departmental Policies, 2020.